ATHANASIOS GOURDOMICHALIS
Java Programms
App7.java

Write an application that initially creates a bank account as an object of the **Account** class (folder ***6. bank accounts CLASS IMPLEMENTATION - deposit - withdraw - balance***). The account details (account number, customer name, and initial balance) are provided by the user, for example:

```
Creating an account
Account Number  : GR100
Customer Name   : MARY JANE
Initial balance : 1000
```

Then, display an options menu through which the user selects one of the following operations (deposit, withdraw, display account details, add interest). All operations apply to the bank account already created:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------

>
```

The implementation of the above operations must be handled by the program using (calling) the corresponding methods of the **Account** class (folder ***6. bank accounts CLASS IMPLEMENTATION - deposit - withdraw - balance***).

Below are examples of how the program should run (user input is shown in bold each time):

Case 1: The user selects deposit and enters a positive amount, for example:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 1
Deposit ...
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1000,00
Amount: 100
New Balance    : 1100,00
```

Case 2: The user selects deposit and enters a negative amount or zero, for example:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 1
Deposit ...
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1100,00
Amount: -100
Error: Deposit Amount is invalid
New Balance    : 1100,00
```

Case 3: The user selects withdrawal and enters a positive amount, for example:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 2
Withdraw ...
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1100,00
Amount: 80
New Balance    : 1020,00
```

Case 4: The user selects withdrawal and enters a negative amount or zero, for example:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 2
Withdraw ...
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1020,00
Amount: -100
Error: Withdraw Amount is invalid
New Balance    : 1020,00
```

Case 5: The user selects withdrawal and enters an amount greater than the available account balance, for example:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 2
Withdraw ...
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1020,00
Amount: 1500
Error: Insufficient funds
New Balance    : 1020,00
```

Case 6: The user selects to view the account details:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 3
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1020,00
```

Case 7: The user selects calculating and adding interest to the account balance:

```
---------------
1. Deposit
2. Withdraw
3. Account Details
4. Add Interest
0. Exit
---------------
> 4
```

```
Add Interest
Account Number : GR100
Customer Name  : MARY JANE
Balance        : 1020,00
New Balance    : 1035,30
```

Important Note: The execution of the program and its input/output must follow the exact format shown above (Cases 1–7). The output formatting must match the examples (note that the account balance is displayed with two decimal places).

Complete the Java code in the **App7.java** file.

Every program must compile and run from the command line (not through an IDE).